Casting Defect Detection — Deep Learning MLOps Technical Report

1. Business problem & objective

Manual visual inspection of manufactured castings is slow, costly and inconsistent. The objective is a reproducible MLOps system that classifies submersible-pump impeller castings as OK or Defective from an image, and that can be trained, evaluated, monitored and retrained as camera and lighting conditions drift. The headline metric is recall on the defect class because a missed defect ships a bad part.

Success criteria:

- Reproducible data preparation and versioned dataset splits.
- Transfer learning using ResNet18.
- MLflow experiment tracking and model registration.
- Statistical, embedding and confidence drift monitoring.
- Drift-triggered retraining with candidate model comparison.

---

2. Dataset

Kaggle *Casting Product Image Data* — approximately 7,348 grayscale 300×300 JPEG images containing two classes (ok_front and def_front), with Defect treated as the positive class.

Data quality is validated before dataset splitting by checking:

- Missing or corrupt images.
- Duplicate images using MD5 hashing.
- Image dimensions.
- Class distribution.

After validation, reproducible versioned train, validation and test splits are generated and stored together with split metadata.

---

3. Methodology & MLOps architecture

data_prep (quality validation · versioned splits · preprocessing · augmentation)

→ ResNet18 transfer learning

→ MLflow tracking and model registry

→ Model evaluation

→ Reference baseline generation

→ Monitoring: statistical drift (Evidently + PSI) + embedding drift + confidence monitoring

→ Drift-triggered retraining

→ Candidate versus production model comparison

Stack

- Python 3.11
- PyTorch
- Torchvision
- MLflow
- Evidently
- NumPy
- Pandas
- Pillow
- Matplotlib

Reproducibility

- Fixed random seed.
- Versioned dataset splits.
- Saved evaluation metrics and model artefacts.
- MLflow experiment tracking.

---

4. Data quality & leakage prevention

1. Duplicate detection using MD5 hashing is performed before dataset splitting to prevent duplicate images from appearing across different dataset splits.
2. Versioned train, validation and test splits are generated and stored together with split metadata to support reproducible experiments.

3. Data augmentation is applied only to the training dataset, while validation and test datasets use deterministic preprocessing.
4. Images are converted from grayscale to three channels and normalised using ImageNet statistics to match the pretrained ResNet18 backbone.

---

5. EDA findings

- The dataset contains two image classes representing OK and Defective castings.
- Dataset quality validation confirms image integrity before model training.
- Representative casting images are visualised for inspection.
- Dataset split statistics are generated for the train, validation and test datasets.
- Image characteristics including brightness, contrast, edge density, sharpness and mean intensity are extracted and later used for statistical drift monitoring.

---

6. Model development

Transfer learning is implemented using ResNet18 with ImageNet pretrained weights. The pretrained backbone is frozen, and the final fully connected layer is replaced with a two-class classification head.

Training configuration:

- ImageNet pretrained ResNet18 backbone.
- Frozen feature extractor.
- Two-class classification head.
- Adam optimiser.
- Class-weighted CrossEntropy Loss.
- Early stopping based on validation F1-score.
- Fixed random seed for reproducibility.

The training workflow consists of:

- Loading versioned dataset splits.
- Building PyTorch DataLoaders.
- Training the classification head.

- Evaluating on the validation dataset after every epoch.
- Logging parameters and metrics using MLflow.
- Saving the best-performing model.
- Evaluating the final model on the test dataset.
- Saving model artefacts, evaluation metrics, reference image features and reference embeddings for the monitoring stage.

7. Evaluation

Defect is the positive class, so recall on defects is the headline metric.

Metric (Test, 715 images)	**Value**
Accuracy	0.9315
Recall (Defect)	0.9316
Precision (Defect)	0.9591
F1 (Defect)	0.9451
Macro F1	0.9269
ROC-AUC	0.9819
Confusion Matrix [[TN, FP], [FN, TP]]	[[244, 18], [31, 422]]

Metric choice is problem-specific. Recall and F1-score are prioritised because a missed defect (false negative) is the most costly error in the casting inspection process.

---

8. MLflow tracking & registry evidence

Runs are logged to the casting_defect_detection experiment, including training parameters, per-epoch metrics, final evaluation metrics and the trained model. The model is registered in the MLflow Model Registry, and model metadata together with evaluation metrics are stored as project artefacts.

Evidence: The MLflow experiment runs and registered model are shown in *Model_Development_and_Tracking.ipynb*

---

9. Monitoring & drift detection (statistical + embedding)

A simulated current production batch, created by applying camera and lighting degradation to held-out test images, is compared with a clean validation reference dataset.

(a) Statistical drift

Per-image features including brightness, contrast, edge density, sharpness and mean intensity are extracted from the reference and current datasets.

Statistical drift is evaluated using Evidently DataDriftPreset together with Population Stability Index (PSI). The monitoring workflow generates:

- Dataset drift status.
- Number of drifted features.
- Drift share.
- Feature-wise PSI values.

(b) Embedding drift (feature-space)

Embedding drift is measured using the 512-dimensional feature representation extracted from the penultimate layer of the trained ResNet18 model.

The workflow consists of:

1. Feature extraction using the trained embedding extractor.
2. Embedding generation for the reference and current datasets.
3. Distance-to-reference-centroid calculation.
4. PSI computation between the two embedding-distance distributions.

Embedding PSI is used to determine whether feature-space drift has occurred.

(c) Confidence monitoring

Average prediction confidence is compared between the reference and current datasets.

The monitoring stage records:

- Mean reference confidence.
- Mean current confidence.
- Confidence drop.
- Confidence alert status.

The monitoring process generates both *drift_report.html* and *drift_summary.json*.

---

10. Retraining & model governance

When the configured drift trigger is satisfied, the retraining workflow:

- Loads the production model.
- Evaluates production performance on a drifted evaluation dataset.
- Trains a drift-augmented candidate model.
- Evaluates the candidate model on the same drifted dataset.
- Compares candidate and production F1-score.
- Promotes the candidate model if it satisfies the configured improvement threshold.
- Otherwise retains the existing production model.
- Saves the retraining decision together with evaluation results and model version information.

---

11. Logging & governance

- Evaluation metrics are stored as project artefacts.
- Dataset split metadata supports reproducible experimentation.
- Drift monitoring summaries are generated after each monitoring run.
- Retraining decisions are stored for auditability.
- MLflow maintains experiment history together with registered model versions.
- Fixed random seeds and versioned dataset splits improve reproducibility.

---

12. Stage-3/4 operational evidence

Because the platform accepts only submitted files, each operation is evidenced inside the notebooks and this report.

Operation	Evidence Captured In
MLflow experiment tracking and Model Registry	Model_Development_and_Tracking.ipynb
Model evaluation	Model_Development_and_Tracking.ipynb
Statistical drift monitoring	Operations_Monitoring_and_Evidence.ipynb
Embedding drift monitoring	Operations_Monitoring_and_Evidence.ipynb
Confidence monitoring	Operations_Monitoring_and_Evidence.ipynb
Retraining decision	Operations_Monitoring_and_Evidence.ipynb

Evidence Figures

MLflow experiment runs

[Insert Screenshot Here]

MLflow Model Registry

[Insert Screenshot Here]

Statistical drift monitoring

[Insert Screenshot Here]

Embedding drift monitoring

[Insert Screenshot Here]

13. Conclusions & recommendations

- A reproducible Deep Learning MLOps pipeline was developed for casting defect classification using transfer learning with ResNet18.
- The workflow includes data preparation, model training, MLflow tracking, model evaluation, drift monitoring and retraining.
- The model achieved an accuracy of 0.9315, recall of 0.9316, F1-score of 0.9451 and ROC-AUC of 0.9819 on the test dataset.

- Versioned dataset splits, stored artefacts and MLflow experiment tracking support reproducibility.

Recommendations

- Extend monitoring to real production data.
- Periodically retrain the model using newly labelled images.
- Fine-tune monitoring thresholds using production data.